Centro Universitário de Excelência
Sistemas de Informação

OAT 3 - MecâniQA João Pessoa

Autores: Arthur de Aquino Anjos
Henrique Estrela Santos
Luan Vinicius Miranda da Silva
Matheus Kaick Rocha da Fonseca
Rafael Pinto Germano Pereira

Feira de Santana, 2026

Agenda

Superar o baseline com um modelo de Machine Learning tunado e provar o ganho com MAE, RMSE e MAPE.

1. Modelos clássicos:
Regressão Linear e Árvore de Decisão com fit() e predict()

2. Grid Search:
Busca exaustiva com TimeSeriesSplit

3. Random Search:
Eficiência de tempo contra o Grid Search

4. Comparação final:
Baseline, Regressão Linear, árvore inicial e árvore tunada

Contexto

- **De onde partimos**
  - A MecâniQA é uma empresa que atende oficinas e auto centers de Feira de Santana e região
  - Nas OAT 1 e 2, os dados foram tratados e criados baselines (Naive e Médias Móveis) para gerar previsões iniciais de manutenção.
- **O desafio desta sprint**
  - Desenvolver, otimizar e comparar modelos de Machine Learning, ajustando códigos e hiperparâmetros para alcançar a maior precisão possível e reduzir erros nas previsões.

Modelos Clássicos

- Regressão Linear e Árvore de Decisão, com parâmetros padrão.
- 5 features: lags de 1, 7 e 30 dias e médias móveis de 7 e 30 dias, calculadas apenas com dados anteriores.
- fit() somente no treino e predict() no teste, utilizando 5 folds cronológicos.
- Os dois modelos apresentam resultados melhores que os baselines em RMSE e MAPE, considerando a média dos cinco folds.

Modelo	RMSE	MAPE (%)
Naive (baseline)	10,99	50,75
MM7 (baseline)	8,10	37,51
MM30 (baseline)	8,18	37,56
Reg. Linear	5,31	28,97
Árvore	7,15	29,59

Médias dos 5 folds (seção 4.3 do notebook). Na OAT 1 os baselines foram só plotados; aqui ganham métricas.

Código dos Modelos

```python
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor

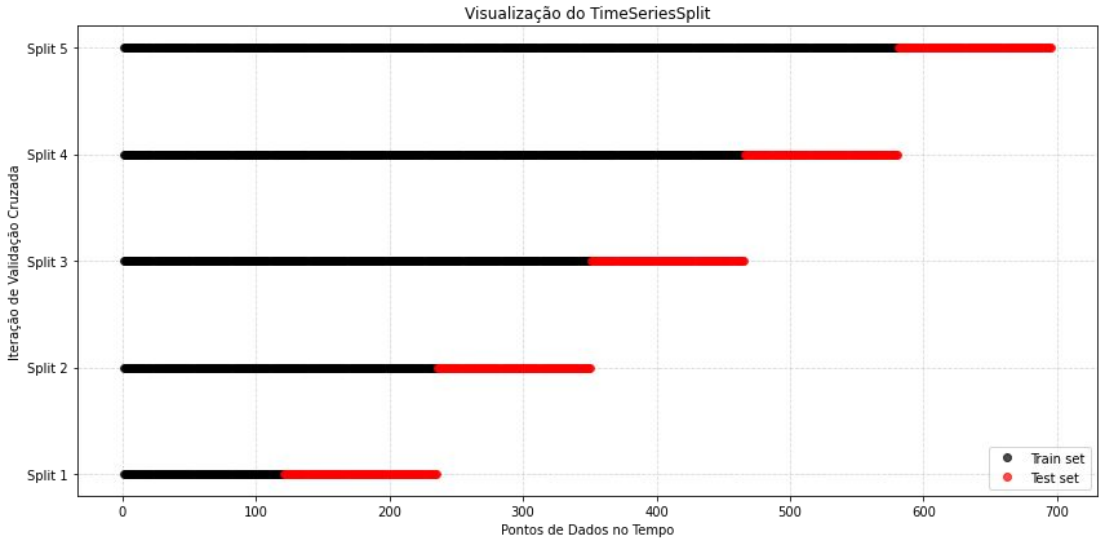
colunas_features = [
    'Trocas_Oleo_Lag_1', 'Trocas_Oleo_Lag_7', 'Trocas_Oleo_Lag_30',
    'Trocas_Oleo_Rolling_Mean_7', 'Trocas_Oleo_Rolling_Mean_30',
]
X = dados_com_features[colunas_features]
y = dados_com_features['Trocas_Oleo']
tscv_regressao = TimeSeriesSplit(n_splits=5)

for nome, classe_modelo in [('Regressao Linear', LinearRegression),
                            ('Arvore de Decisao', DecisionTreeRegressor)]:
    for train_index, test_index in tscv_regressao.split(X):
        modelo = classe_modelo()          # hiperparâmetros default
        modelo.fit(X.iloc[train_index], y.iloc[train_index])
        y_pred = modelo.predict(X.iloc[test_index])
```

Modelos com hiperparâmetros padrão: fit() no treino e predict() no teste de cada fold.

Grid Search

- Grid de 12 combinações de profundidade máxima e tamanho mínimo da folha
- GridSearchCV com TimeSeriesSplit de 5 folds, otimizando o MAPE
- A validação respeita a linha do tempo: o modelo não pode ver o futuro para prever o passado



Treino (preto) sempre antes do teste (vermelho).

Código do Grid Search

```python
parametros_arvore = {
    'max_depth': [2, 3, 5, None],
    'min_samples_leaf': [1, 3, 5],
}

cv_grid = TimeSeriesSplit(n_splits=5)
grid_search = GridSearchCV(
    estimator=DecisionTreeRegressor(random_state=42),
    param_grid=parametros_arvore,
    scoring=scorer_mape_sem_zeros,   # -MAPE (sem alvos = 0)
    cv=cv_grid,
    refit=True,
    n_jobs=1,
)
grid_search.fit(X, y)
print('best_params_:', grid_search.best_params_)
```

GridSearchCV integrado ao TimeSeriesSplit(n_splits=5), com scorer de MAPE.

Resultado do Grid Search

```
best_params_: {'max_depth': 3, 'min_samples_leaf': 1}
MAPE medio da arvore padrao: 29.59%
MAPE medio da melhor configuracao (GridSearchCV): 26.95%
Redução do MAPE vs. árvore padrão: 8.95%
Wall time: 583 ms
```

- Melhor configuração: profundidade máxima 3 e folha mínima 1
- 12 combinações × 5 folds = 60 treinos
- MAPE: de 29,59% (árvore padrão) para 26,95%, uma redução de 8,95%

Random Search

```
inicio_random = time.perf_counter()
busca_aleatoria = RandomizedSearchCV(
    estimator=arvore_fixa,
    param_distributions=parametros_arvore,
    n_iter=6,                          # 6 das 12 combinações
    scoring=scorer_mape_sem_zeros,
    cv=cv_comparacao,                  # mesmo TimeSeriesSplit
    random_state=42,
    refit=True,
    n_jobs=1,
)
busca_aleatoria.fit(X, y)
tempo_random = time.perf_counter() - inicio_random
```

RandomizedSearchCV com n_iter=6 (6 das 12 combinações), mesmo TimeSeriesSplit e mesma métrica. Tempo medido com time.perf_counter() e %%time, com n_jobs=1.

Grid vs Random

Busca	Combinações	Tempo (s)	MAPE (%)	Melhores parâmetros
Grid Search	12	0,574	26,95	max_depth=3, min_samples_leaf=1
Random Search	6	0,287	27,77	max_depth=2, min_samples_leaf=1

- Custo: o Random economizou 0,287 s (cerca de 50% do tempo do Grid)
- Benefício perdido: MAPE 0,83 ponto percentual maior que o do Grid
- Trade-off: com grid pequeno o Grid compensa e venceu; em grades grandes, o Random economiza tempo

Tempos de uma execução; variam conforme o computador. O 50% é cálculo sobre os tempos medidos.

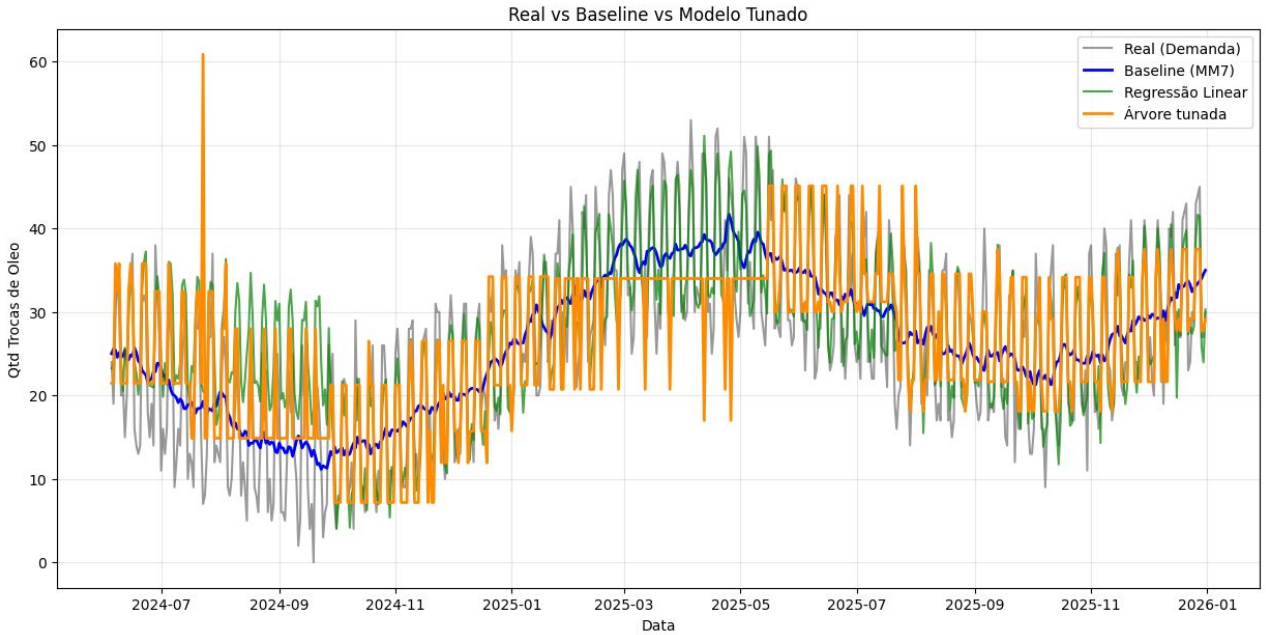
Comparação Final

Cenário	MAE	RMSE	MAPE (%)
Baseline MM7	7,37	8,11	37,47
Regressão Linear	4,44	5,75	28,87
Árvore inicial (padrão)	7,76	11,14	34,67
Árvore tunada	**5,02**	**6,74**	**26,88**

- Baseline: usamos a MM7, pois foi a melhor da OAT 1 em RMSE e MAPE.
- Contra o baseline: a árvore tunada errou menos nas três métricas (queda de 17% a 32%).
- Contra a Regressão Linear: a árvore vence no MAPE, e a Regressão vence no MAE e no RMSE.
- Comparação justa: mesmos 5 folds cronológicos, só em blocos de teste.

Nota: aqui as métricas são calculadas sobre os blocos de teste juntos; nos slides anteriores, são médias dos folds.

Gráfico Comparativo



Trocas de óleo reais (cinza), baseline MM7 (azul), Regressão Linear (verde) e árvore tunada (laranja), nos blocos de teste dos 5 folds.

Considerações finais

- A árvore tunada superou o baseline MM7 nas três métricas (MAE, RMSE e MAPE), com redução de erro entre 17% e 32%.
- O ajuste de hiperparâmetros melhorou bastante a árvore em relação à versão com parâmetros padrão.
- A Regressão Linear teve menor MAE e RMSE que a árvore tunada, e a árvore tunada teve o menor MAPE. Nenhum dos dois modelos ganhou em tudo.
- O Grid Search chegou ao menor erro, e o Random Search foi cerca de duas vezes mais rápido, com um erro um pouco maior.
- Toda a validação foi cronológica, sem uso de dados do futuro.

Referências

- AMAZON WEB SERVICES (AWS). **Métricas de avaliação de previsões**. Disponível em: https://docs.aws.amazon.com/pt_br/forecast/latest/dg/metrics.html
- SATHEESH, Vishnu. **Hyper parameter tuning (GridSearchCV vs RandomizedSearchCV)**. Disponível em: https://medium.com/analytics-vidhya/hyper-parameter-tuning-gridsearchcv-vs-randomizedsearchcv-499862e3ca5
- GOOGLE FOR DEVELOPERS. **Florestas aleatórias**. Disponível em: https://developers.google.com/machine-learning/decision-forests/random-forests?hl=pt-br.
- IBM. **O que é o scikit-learn?**. Disponível em: https://www.ibm.com/br-pt/think/topics/scikit-learn.
- SCIKIT-LEARN. **Tuning the hyper-parameters of an estimator**. Disponível em: https://scikit-learn.org/stable/modules/grid_search.html.